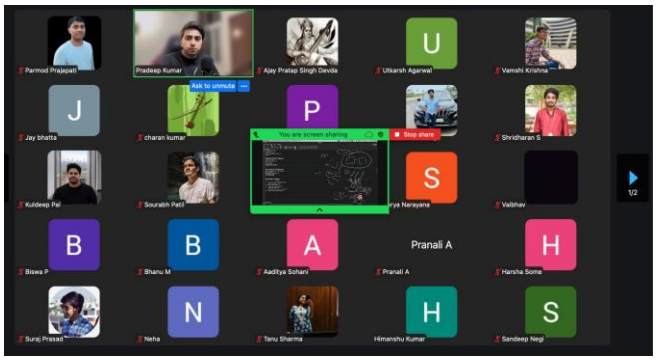

I WILL
MENTOR
YOU
AND
GUIDE
YOU
TO CRACK JOB IN
2026
PERSONALIZED MENTORSHIP
EXPERT CAREER GUIDANCE
SKILL BUILDING & STRATEGY
INTERVIEW PREPARATION
YOUR SUCCESS IS MY MISSION
Your Success is My Mission
Let's make 2026 your breakthough year!


Pranali A



New Batch Enrollments started
livecohortbypradeep.com

System Design & DSA

Complete Study Guide

HLD · LLD · DSA — 150+ Questions

Covers

High-Level Design (HLD)	13 concept categories + classic system design problems
Low-Level Design (LLD)	OOP, SOLID, Design Patterns (Creational / Structural / Behavioral), real-world problems
Data Structures & Algorithms	20 patterns · 150+ curated LeetCode-style problems

Prepared for students as a comprehensive reference for interview preparation and deep learning.

PART 1

High-Level Design (HLD)

High-Level Design focuses on the architecture of large-scale distributed systems. Master these concepts to design systems that are scalable, reliable, and maintainable.

1. Fundamentals & Core Concepts

- Client-Server Architecture
- Monolith vs Microservices
- Synchronous vs Asynchronous Communication
- Latency vs Throughput vs Availability
- CAP Theorem (Consistency, Availability, Partition Tolerance)
- PACELC Theorem
- SLA / SLO / SLI

2. Scalability

- Horizontal vs Vertical Scaling
- Load Balancing (L4 vs L7, Round Robin, Least Connections)
- Auto-scaling Strategies
- Stateless vs Stateful Services

3. Databases

- SQL vs NoSQL (when to use what)
- Database Sharding & Partitioning
- Replication (Master-Slave, Master-Master)
- Indexing Strategies
- ACID vs BASE Properties
- Read Replicas
- Consistent Hashing
- NewSQL (CockroachDB, Spanner)

4. Caching

- Cache Strategies (Write-through, Write-back, Write-around)
- Cache Eviction Policies (LRU, LFU, TTL)
- Distributed Caching (Redis, Memcached)
- CDN (Content Delivery Network)

- Cache Invalidation Strategies
- Cache Stampede / Thundering Herd

5. Messaging & Event-Driven Systems

- Message Queues (RabbitMQ, SQS)
- Event Streaming (Kafka, Kinesis)
- Pub/Sub Pattern
- Dead Letter Queues
- Exactly-once vs At-least-once Delivery
- Event Sourcing & CQRS

6. APIs & Communication

- REST vs GraphQL vs gRPC
- API Gateway
- Rate Limiting & Throttling
- Pagination Strategies (Cursor, Offset)
- Webhooks vs Polling vs Long-polling vs SSE vs WebSockets

7. Search

- Full-text Search (Elasticsearch, Solr)
- Inverted Index
- Search Ranking & Relevance
- Typeahead / Autocomplete Design

8. Storage & File Systems

- Object Storage (S3, GCS)
- Block vs Object vs File Storage
- Blob Storage Design
- Large File Upload Strategies (Chunked, Multipart)

9. Security & Auth

- Authentication vs Authorization
- OAuth 2.0 / OpenID Connect / JWT
- API Key Management
- TLS / HTTPS
- Rate Limiting for Abuse Prevention

10. Observability

- Logging (Structured Logs, Centralized Logging)
- Metrics & Monitoring (Prometheus, Grafana)
- Distributed Tracing (Jaeger, Zipkin, OpenTelemetry)
- Alerting Strategies
- Health Checks & Circuit Breaker Pattern

11. Distributed Systems Concepts

- Consensus Algorithms (Raft, Paxos)
- Distributed Locks (Redlock)
- Leader Election
- Two-Phase Commit (2PC)
- Saga Pattern (for Distributed Transactions)
- Idempotency

12. Reliability & Fault Tolerance

- Circuit Breaker Pattern
- Retry with Exponential Backoff
- Bulkhead Pattern
- Graceful Degradation
- Chaos Engineering Concepts
- Disaster Recovery (RPO & RTO)

13. Infrastructure & Deployment

- Docker & Kubernetes Basics (for design context)
- Service Mesh (Istio)
- Blue-Green / Canary Deployments
- DNS & Anycast Routing
- Reverse Proxy (Nginx)

System Design Problems

Practice these classic problems end-to-end. For each, define requirements, estimate scale, draw the high-level architecture, then drill into key components.

Category	Problems
Social	Twitter Feed, Instagram, Facebook Messenger
Storage	Google Drive, Dropbox, YouTube
Commerce	Uber, Zomato, BookMyShow
Finance	Payment System, Wallet, Rate Limiter
Search	Google Search, Typeahead
Infra	URL Shortener, Notification System, Web Crawler
Real-time	Slack / WhatsApp Chat, Live Score System

PART 2

Low-Level Design (LLD)

Low-Level Design tests your ability to translate requirements into clean, extensible object-oriented code. Focus on identifying the right classes, applying design patterns, and writing SOLID code.

1. OOP Fundamentals

- Encapsulation
- Abstraction
- Inheritance
- Polymorphism

2. Class Relationships & UML

- Association
- Aggregation
- Composition

3. SOLID Principles with Code Examples

- Single Responsibility Principle (SRP)
- Open Closed Principle (OCP)
- Liskov Substitution Principle (LSP)
- Interface Segregation Principle (ISP)
- Dependency Inversion Principle (DIP)

Design Patterns

4. Creational Design Patterns

- Factory Pattern
- Abstract Factory Pattern
- Builder Pattern
- Singleton Pattern (Thread-safe)
- Real-world use cases in production systems
- Refactoring bad code using design patterns

5. Structural Design Patterns

- Adapter Pattern
- Decorator Pattern
- Facade Pattern
- Proxy Pattern
- Flyweight Pattern
- Composite Design Pattern
- How to extend functionality without modifying code
- Real interview examples using structural patterns

6. Behavioral Design Patterns

- Strategy Pattern
- Observer Pattern
- State Pattern
- Chain of responsibility
- Writing extensible and flexible systems

Real-World System Problems

Classic Game & Simulation Systems

Focus: Identifying classes correctly · Applying design patterns · Writing extensible code

- Design Parking Lot System
- Design Elevator System
- Design Tic-Tac-Toe
- Design Snake and Ladder

Concurrent & Thread-safe Systems

Focus: Thread-safety · Concurrent access handling · Extensible system design

- Design Rate Limiter
- Design Cache (LRU Cache)
- Design Logger System
- Design File System

Business & Domain Systems

Focus: Domain modelling · Real-world entity relationships · Handling edge cases

- Design Notification System
- Design Splitwise
- Design ATM System
- Design Vending Machine

More LLD Problems

Focus: End-to-end design from scratch

- URL Shortener
- Vending Machine
- Inventory Management
- Payment Gateway
- Load Balancer
- Pub-Sub System
- BookMyShow
- Amazon Locker Service
- Ride Booking Service
- Trading Platform
- Music Streaming

Additional LLD Problems

Focus: Comprehensive practice set

- Design Tic-Tac-Toe
- Design Snake and Ladder Game
- Design LRU Cache
- Design Bloom Filter
- Design Parking Lot
- Design Search Autocomplete System
- Design Task Management System
- Design Notification System
- Design Coffee Vending Machine
- Design Restaurant Management System
- Design Online Stock Exchange
- Design Movie Booking System
- Design Online Auction System

Mock Interviews: Conduct 2–3 mock interviews with peers. Simulate real interview conditions — 45 minutes, whiteboard or Google Docs, and debrief together afterward.

PART 3

Data Structures & Algorithms (DSA) — 150+ Questions

Master these 20 patterns to solve 90% of interview problems. Each pattern represents a core technique — understand the pattern first, then practice the problems.

1. Arrays

- Move Zeroes
- Product of Array Except Self
- Rotate Array
- Merge Intervals
- Insert Interval
- Game of Life
- 132 Pattern
- First Missing Positive (Cycling Sort)
- LC 287 – Find Duplicate Number
- LC 448 – Find All Numbers Disappeared
- LC 645 – Set Mismatch
- LC 442 – Find All Duplicates

2. Two Pointers

- Two Sum II
- Container With Most Water
- 3Sum
- Remove Duplicates from Sorted Array
- Valid Palindrome

3. Sliding Window

- Longest Substring Without Repeating Characters
- Minimum Window Substring
- Permutation in String
- Find All Anagrams in a String
- Maximum Average Subarray

4. Prefix Sum

- Subarray Sum Equals K
- Continuous Subarray Sum

- Range Sum Query
- Product of Array Except Self

5. Hash Map / Hashing

- Two Sum
- Group Anagrams
- Isomorphic Strings
- Happy Number

6. Fast & Slow Pointers (Cycle Detection)

- Linked List Cycle
- Linked List Cycle II
- Happy Number
- Find Duplicate Number

7. Linked List

- Reverse Linked List
- Merge Two Sorted Lists
- Remove Nth Node from End
- Add Two Numbers
- Copy List with Random Pointer
- Swap Nodes in Pairs
- Rotate List

8. Binary Search

- Binary Search
- Search in Rotated Sorted Array
- Find Peak Element
- Median of Two Sorted Arrays

9. Stack / Monotonic Stack

- Valid Parentheses
- Daily Temperatures
- Next Greater Element
- Largest Rectangle in Histogram
- 132 Pattern

10. Heap / Top K Elements

- Top K Frequent Elements
- K Closest Points to Origin
- Find Median from Data Stream
- Merge K Sorted Lists

11. Backtracking

- Subsets
- Permutations
- Combination Sum
- Word Search
- N-Queens

12. Trees (DFS)

- Maximum Depth of Binary Tree
- Path Sum
- Diameter of Binary Tree
- Lowest Common Ancestor

13. Trees (BFS)

- Binary Tree Level Order Traversal
- Right Side View
- Minimum Depth of Binary Tree

14. Binary Search Tree

- Validate BST
- Kth Smallest Element in BST
- Convert Sorted Array to BST

15. Trie

- Implement Trie
- Word Search II
- Design Add and Search Words

16. Graphs

- Number of Islands
- Clone Graph
- Course Schedule
- Pacific Atlantic Water Flow

17. Union Find

- Number of Connected Components
- Redundant Connection
- Accounts Merge

18. Dynamic Programming

- Climbing Stairs
- House Robber
- Longest Increasing Subsequence
- Coin Change
- Edit Distance

19. Greedy

- Jump Game
- Gas Station
- Partition Labels

20. Bit Manipulation

- Single Number
- Counting Bits
- Reverse Bits

Study Tip: For each pattern, (1) understand the core idea, (2) solve 2–3 easy problems, (3) attempt medium problems, (4) time yourself (aim for 20–25 min per medium problem). Track your weak patterns and revisit them weekly.

Good luck — consistency beats intensity. Solve one pattern per day and you'll be interview-ready in under a month.

Additional Resources and repository of all the resources
https://docs.google.com/document/d/1EB1WlFVIPCJHpH4HEHDyTvEmg9QY4YzlY5IG8xrhk6I/edit?tab=t.0